Informe IV (Arquitectura del Computador)

Jesus Contreras (C.I: 31.021.750)

2026

1. Preguntas

1.1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).

La Figura 1 muestra el ciclo completo de una interrupción de hardware, desde que ocurre el evento hasta que se atiende:

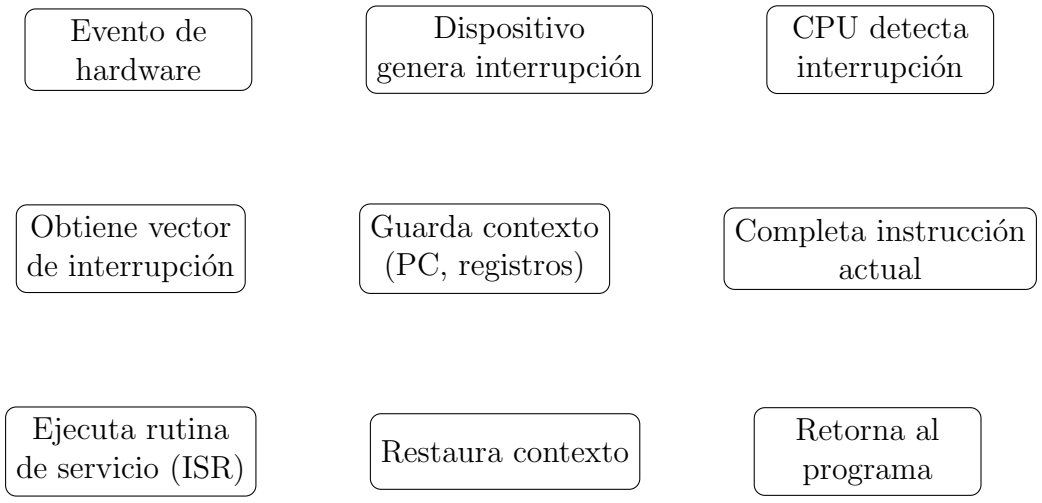


Figura 1: Ciclo completo de una interrupción de hardware

1.2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

1.3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

Las interrupciones ofrecen ventajas significativas en la eficiencia del uso del procesador:

Característica	Sondeo (Polling)	Interrupciones
Mecanismo	CPU verifica constantemente el estado de los dispositivos	Dispositivo notifica a CPU cuando necesita atención
Eficiencia CPU	Baja - desperdicia ciclos verificando dispositivos inactivos	Alta - CPU hace otras tareas hasta ser notificada
Latencia de respuesta	Depende de la frecuencia de sondeo	Generalmente más baja y predecible
Complejidad	Simple de implementar	Más complejo (hardware y software)
Escalabilidad	Mala - más dispositivos requieren más sondeos	Buena - dispositivos adicionales no aumentan carga
Aplicaciones típicas	Sistemas simples, tiempo real crítico	Sistemas generales, multitarea

Cuadro 1: Comparación entre sondeo e interrupciones

- **Multitarea eficiente:** La CPU puede ejecutar programas de usuario mientras los dispositivos operan en paralelo, solo interrumpiendo cuando es necesario.
- **Reducción de esperas activas:** Elimina los ciclos desperdiciados en verificaciones constantes de estado de dispositivos.
- **Mejor aprovechamiento del pipeline:** Al no interrumpir constantemente para verificar dispositivos, el pipeline del procesador se mantiene lleno con instrucciones útiles.
- **Capacidad de respuesta:** Eventos urgentes pueden ser atendidos inmediatamente sin esperar al siguiente ciclo de sondeo.
- **Escalabilidad:** El rendimiento del sistema no se degrada significativamente al añadir más dispositivos, ya que la CPU solo se ocupa cuando hay eventos reales.

1.4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

MIPS32 utiliza los siguientes registros CP0 (Co-processador 0) para la gestión de interrupciones:

- **Status (SR):** Controla el estado del procesador y la habilitación de interrupciones. Bits importantes:
  - IE (bit 0): Habilita/deshabilita interrupciones globalmente
  - IM (bits 8-15): Máscara de interrupciones por hardware
  - KSU (bits 3-2): Modo de operación (kernel/supervisor/usuario)

- **Cause:** Indica la causa de la última excepción o interrupción:
  - ExcCode (bits 2-6): Código de la excepción
  - IP (bits 8-15): Pending interrupts (interrupciones pendientes)
- **EPC (Exception Program Counter):** Guarda la dirección de la instrucción donde ocurrió la excepción o donde se debe reanudar la ejecución.
- **PRId (Processor ID):** Identifica el tipo de procesador.
- **Compare y Count:** Utilizados para temporizadores e interrupciones de reloj.

1.5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

Es necesario guardar el contexto (registros) al entrar en una rutina de servicio de interrupción (ISR) porque:

- **Preservación del estado:** La ISR utilizará los registros del procesador para su ejecución, sobrescribiendo su contenido.
- **Transparencia:** El programa interrumpido debe poder continuar su ejecución como si nada hubiera ocurrido, sin percibir que fue interrumpido.
- **Reentrancia:** Permite que múltiples interrupciones anidadas puedan manejarse correctamente sin corromper el estado de programas interrumpidos.
- **Registros específicos:** Además de los registros de propósito general, es crucial guardar registros como EPC y Status para poder reanudar correctamente la ejecución.

1.6. Momentos en que pueden generarse excepciones en un sistema MIPS32.

1.6.1. a) Enumera al menos 4 situaciones en las que se pueda generar una excepción (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.).

1. **Desbordamiento aritmético (Overflow):** Ocurre cuando una operación aritmética produce un resultado fuera del rango representable.
2. **Fallo de dirección (Address error):** Acceso a memoria con una dirección mal alineada o fuera de los límites permitidos.
3. **Instrucción no válida (Reserved instruction):** Intento de ejecutar un código de operación no implementado en el procesador.
4. **System call:** Instrucción `syscall` ejecutada explícitamente por el programa para solicitar servicios del sistema operativo.

1.6.2. b) Explica qué etapas del pipeline pueden provocar una excepción y por qué.

Las diferentes etapas del pipeline pueden generar excepciones:

- **IF (Instruction Fetch):**
  - Error de dirección en la búsqueda de instrucción
  - Fallo de TLB en instrucción
  - Instrucción desde dirección no válida
- **ID (Instruction Decode):**
  - Instrucción no válida (reserved instruction)
  - Instrucción privilegiada en modo usuario
  - Syscall y Break (detectadas en decodificación)
- **EX (Execute):**
  - Desbordamiento aritmético (overflow)
  - Trampas condicionales
- **MEM (Memory Access):**
  - Error de alineación en acceso a memoria
  - Fallo de TLB en datos
  - Error de dirección en carga/almacenamiento
- **WB (Write Back):** Generalmente no genera excepciones nuevas, pero pueden propagarse excepciones de etapas anteriores.

1.7. Estrategias de tratamiento de excepciones e interrupciones

1.7.1. a) Explica las diferencias entre interrupciones y excepciones (¿son síncronas o asíncronas?).

1.7.2. b) Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32 ¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

Estrategia de vector único:

- Todas las excepciones e interrupciones redirigen la ejecución a una dirección fija (generalmente 0x80000180)
- El software debe examinar el registro `Cause` para determinar la causa
- Más simple en hardware pero requiere software más complejo

Característica	Excepciones	Interrupciones
Sincronía	**Síncronas:** Ocurren en relación directa con la ejecución de instrucciones	**Asíncronas:** Ocurren independientemente de la ejecución del programa
Origen	Internas al procesador o por instrucciones	Externas (dispositivos de hardware)
Predictibilidad	Predecibles (ocurren siempre en mismas condiciones)	Impredecibles (dependen de eventos externos)
Reintentabilidad	Algunas excepciones (como overflow) no permiten reintentar la instrucción	Generalmente se puede reanudar el programa interrumpido

Cuadro 2: Comparación entre excepciones e interrupciones

Estrategia de vectores múltiples:

- Diferentes tipos de excepción tienen diferentes direcciones de servicio
- Hardware redirige directamente a la rutina específica
- Reduce latencia pero requiere más lógica hardware

Redirección a rutina de servicio:

1. El hardware guarda la dirección de retorno en `EPC`
2. El procesador cambia a modo kernel
3. Deshabilita interrupciones (opcional)
4. Salta a la dirección de la rutina de servicio (generalmente desde un registro o dirección fija)

Función del registro EPC: El registro `EPC` (Exception Program Counter) almacena la dirección de la instrucción donde ocurrió la excepción o la instrucción que debe reanudarse después del tratamiento. Para interrupciones, guarda la dirección de la siguiente instrucción a ejecutar. Para excepciones, puede guardar la dirección de la instrucción que causó la excepción o la siguiente, dependiendo del tipo de excepción.

1.8. Habilitación de interrupciones en dispositivos y procesador

1.8.1. a) Explica cómo se habilitan las interrupciones

Teclado:

- Generalmente se habilita mediante un bit de control en el registro de control del controlador de teclado
- Ejemplo: bit Ïnterrupt Enable.[en] el registro de estado del dispositivo

- Configuración típica: escribir 1 en el bit correspondiente para permitir que el teclado genere interrupciones cuando se pulse una tecla

Pantalla:

- Similar al teclado, se configura mediante bits de habilitación en el controlador de pantalla
- Puede tener interrupciones separadas para:
  - VSync (sincronización vertical)
  - Buffer vacío/listo para más datos
  - Finalización de transferencia DMA

Procesador MIPS32 - Registro Status:
Para habilitar interrupciones en el procesador MIPS32, se deben modificar los siguientes bits del registro `Status`:

- **IE (Interrupt Enable) - bit 0:**
  - 0 = Todas las interrupciones deshabilitadas
  - 1 = Interrupciones habilitadas globalmente
- **IM (Interrupt Mask) - bits 8 a 15:**
  - Cada bit corresponde a un nivel de interrupción hardware
  - 1 = Habilita ese nivel específico de interrupción
  - 0 = Deshabilita ese nivel
- **KSU (Kernel/Supervisor/User) - bits 3-2:** Debe estar en modo kernel para modificar registros CP0
- **EXL (Exception Level) - bit 1:** Debe ser 0 para que las interrupciones sean reconocidas

1.8.2. ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?

Si habilitamos interrupciones en los dispositivos pero no en el procesador:

- Los dispositivos generarán señales de interrupción que no serán reconocidas por la CPU
- Las interrupciones quedarán pendientes (pending) pero nunca serán atendidas
- Algunos dispositivos pueden dejar de funcionar correctamente si sus buffers se llenan al no ser servidos
- El sistema puede volverse inestable si se pierden eventos críticos
- No hay ningún beneficio; es una configuración errónea

1.9. Procesamiento de interrupciones

1.9.1. Describe paso a paso qué ocurre cuando se produce una interrupción de reloj:

Hardware:

1. El temporizador de hardware alcanza el valor programado en el registro `Compare`
2. El bit IP (Interrupt Pending) correspondiente en el registro `Cause` se establece
3. Si las interrupciones están habilitadas (IE=1, IM correspondiente=1, EXL=0):
   - El hardware guarda la dirección de la siguiente instrucción en `EPC`
   - Establece EXL=1 en el registro `Status` (indicando que está manejando una excepción)
   - Cambia a modo kernel (KSU=0)
   - Deshabilita interrupciones automáticamente (aunque depende de configuración)
   - Salta a la dirección de la rutina de servicio (0x80000180 en MIPS32)

Software (Rutina de servicio):

1. **Salva contexto:** Guarda registros de propósito general en la pila del kernel
2. **Identifica causa:** Lee registro `Cause` para confirmar interrupción de reloj
3. **Acknowledge:** Reconoce la interrupción al dispositivo (resetea el pending bit)
4. **Ejecuta servicio:** Incrementa contador del sistema, programa siguiente interrupción, etc.
5. **Restaura contexto:** Recupera registros guardados
6. **Retorna:** Ejecuta `eret` (Exception Return):
   - Restaura PC desde `EPC`
   - Limpia EXL
   - Restaura modo anterior
   - Habilita interrupciones (según configuración previa)

1.9.2. ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

Es crucial guardar el contexto (registros generales, EPC, Status) porque:

- **Transparencia:** El programa interrumpido no debe notar que fue interrumpido
- **Consistencia:** Los registros contienen datos parcialmente procesados que serán necesarios al reanudar

- **Reentrancia:** Permite manejar interrupciones anidadas correctamente
- **EPC:** Es la única referencia de dónde reanudar la ejecución
- **Status:** Guarda el estado de privilegio y máscara de interrupciones previo

1.10. Interrupciones de reloj y control de ejecución

1.10.1. a) Explica cómo una interrupción de reloj puede usarse para:

Evitar bucles infinitos:

- El sistema operativo programa interrupciones periódicas de reloj (ej. cada 10ms)
- En cada interrupción, el scheduler verifica cuánto tiempo ha ejecutado el proceso actual
- Si un proceso excede su quantum (time slice), el scheduler fuerza un cambio de contexto
- Esto evita que un bucle infinito (intencional o por error) monopolice la CPU

Finalizar programas que superan tiempo máximo:

- El SO mantiene un contador de tiempo total de ejecución por proceso
- En cada interrupción de reloj, incrementa el contador del proceso actual
- Si el proceso supera su tiempo máximo permitido (timeout), el SO:
  1. Envía una señal de terminación al proceso
  2. Libera sus recursos
  3. Carga el siguiente proceso en la cola de ejecución
- Esto previene que programas maliciosos o con errores consuman recursos indefinidamente

1.10.2. b) ¿Qué debe hacer el software si el programa finaliza antes de que ocurra la interrupción de reloj?

Si un programa finaliza antes de que ocurra la interrupción de reloj:

- El programa realiza una llamada al sistema (`exit` o similar)
- El SO libera todos los recursos del proceso (memoria, archivos abiertos, etc.)
- Cancela cualquier temporizador o interrupción pendiente asociada al proceso
- Actualiza las estructuras de scheduling (elimina el proceso de la cola de listos)
- El scheduler selecciona el siguiente proceso a ejecutar
- La interrupción de reloj programada para ese proceso queda huérfana y debe ser ignorada o reasignada
- El SO debe asegurarse de que el manejador de interrupciones verifique que el proceso actual sigue siendo válido

2. Análisis y Discusión de Resultados

Del análisis realizado sobre interrupciones y excepciones en sistemas MIPS32, se pueden extraer las siguientes conclusiones:

- **Eficiencia vs. Complejidad:** Las interrupciones representan un equilibrio óptimo entre eficiencia del procesador y complejidad del sistema. Aunque introducen complejidad adicional en hardware y software, permiten un aprovechamiento mucho mayor de la CPU comparado con el sondeo.
- **Mecanismos de protección:** La separación entre modos kernel/usuario, junto con registros especiales como Status y EPC, proporciona un marco robusto para la protección del sistema. Las interrupciones y excepciones son el mecanismo fundamental para que el SO mantenga el control sobre los recursos.
- **Contexto y transparencia:** La necesidad de guardar y restaurar el contexto en cada interrupción es crucial para mantener la ilusión de ejecución continua en cada proceso, base de los sistemas multitarea.
- **Control de ejecución:** La interrupción de reloj es una herramienta fundamental para la planificación de procesos (scheduling) y para garantizar que ningún programa pueda monopolizar la CPU, ya sea por error o intencionalmente.
- **Sincronía vs. Asincronía:** La distinción entre excepciones (síncronas) e interrupciones (asíncronas) es fundamental para entender el comportamiento del sistema y cómo deben manejarse cada tipo de evento.
- **Jerarquía de habilitación:** El sistema de habilitación de interrupciones en múltiples niveles (dispositivo, controlador, procesador) permite un control granular sobre qué eventos pueden interrumpir la CPU, esencial para sistemas complejos con múltiples periféricos.

En conclusión, el sistema de interrupciones y excepciones en MIPS32 proporciona un mecanismo elegante y eficiente para la gestión de eventos asíncronos y situaciones excepcionales, permitiendo la construcción de sistemas operativos modernos con multitarea, protección de memoria y tiempo compartido.